

HSEA Homework 4 报告

241220001 寇裕林

目录

一、问题简介

二、基础演化算法的实现与优化

2.1 不同演化算子的设计

2.2 不同演化算子的对比试验

2.3 演化算法的一些优化

2.4 寻找最优演化算法的实验

三、基于强化学习的演化算法

3.1 可学习演化算子的设计与实现

3.2 可学习演化算子的训练推理实验

四、总结

一、问题简介

本次作业研究的是生物多样性曲线优化问题，优化目标是寻找一个事件排序，使其与地层剖面中的化石观测数据尽可能一致。假设有 n 个物种，我们把每一个物种用起始时间和灭绝时间统一编号，得到 $\text{dim} = 2n$ 的排列，把原始问题转化成目标函数黑盒的序列优化问题。该问题还带有约束，例如同一 taxon 的 FAD 应早于 LAD，存在共生关系的 taxon 在时间区间上需要有交叠等等。

本次作业我们用到的主要文件如下：

```
1 | main.py -> 算法的主入口
2 | scripts/run_earth_124*.sh -> 不同算法配置的运行脚本
3 | algorithms/_map_elites.py -> 基础演化算法主代码
4 | algorithms/_ea_operators.py -> 演化算子代码
5 | algorithms/_ea.py -> 基于强化学习的演化算法主代码
6 | algorithms/_utils.py -> 初始化、采样等功能函数代码
7 | algorithms/_constraint_utils.py -> 基于约束的算子代码
8 | train.py -> 强化学习训练代码
9 | results/ -> 实验结果保存目录
10 | multi_env_models/ -> 强化学习模型参数保存目录
11 | benchmarks/CONOPLib_124/*.dat -> 先验约束条件
```

二、基础演化算法的实现与优化

2.1 不同演化算子的设计

本项目中的基础排列算子主要定义在 `algorithms/_ea_operator.py`。由于本问题的解是事件排列，我们使用的 `mutation` 和 `crossover` 算子的核心作用都是改变或重组事件顺序，同时尽量保留已有候选解中可能有用的局部结构。

Mutation 算子。

`swap_mutation` 随机选择两个位置并交换对应事件。它的变化幅度较小，适合在已有解附近做局部调整。实现上只需要复制原排列，然后多次随机交换两个位置。

`insert_mutation` 随机选择一个事件，并将其插入到另一个位置。该算子比较符合事件排序问题的直觉，因为很多时候一个事件需要整体前移或后移，而不是只和相邻事件交换。它会改变被选事件的位置，同时较好保留其他事件的相对顺序。

`reversal_mutation` 随机选择一个连续区间并反转该区间内的事件顺序。它的扰动范围比 `swap` 和 `insert` 更大，可以一次性改变多个事件的相对次序。

`shuffle_mutation` 随机选择若干事件并打乱它们的顺序。该算子可以快速引入较大随机性，主要用于增加搜索多样性。

`shift_mutation` 在局部邻域内移动或交换某个事件。它的扰动幅度较温和，更接近一种局部搜索式操作。

除了这些基础 `mutation`，我们后续还实现了 `adaptive_mix` 和 `constraint_insert`，在后续报告中会详细介绍。

Crossover 算子。

`order_crossover` 保留父代 1 中的一个连续片段，然后按照父代 2 中的相对顺序填充剩余事件。它强调继承相对顺序结构，适合排列问题。

`pmx_crossover` 是部分映射交叉。它先保留一个父代片段，再通过映射关系处理重复元素和冲突。该方法既保留部分位置结构，也保留两个父代之间的元素对应关系。

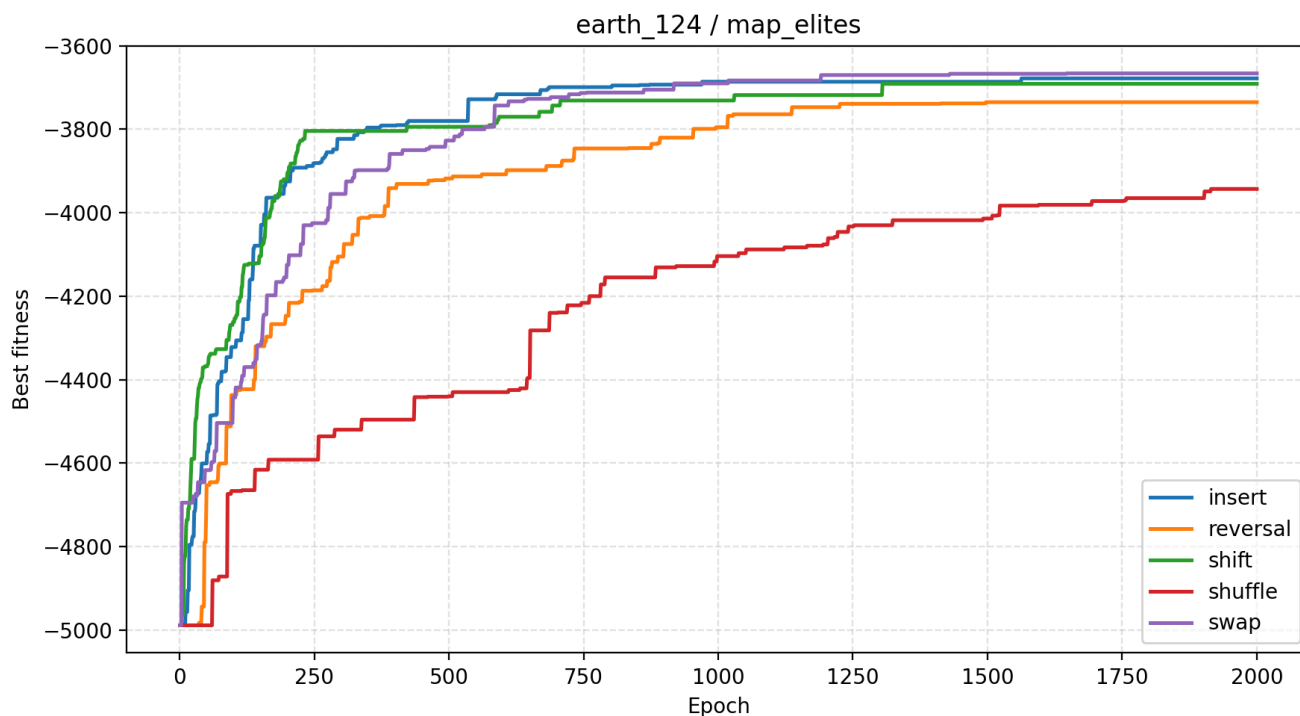
`cycle_crossover` 根据两个父代之间的元素-位置循环来构造子代。它更强调继承父代中的位置循环结构，能够较强烈地保留父代中的绝对位置信息。

2.2 不同演化算子的对比试验

为了比较不同演化算子对 `earth_124` 的影响，我们分别进行了 `mutation` 对比实验和 `crossover` 对比实验。所有实验均使用 MAP-Elites 框架，并开启锦标赛父代选择。锦标赛选择只使用 `archive` 中已经保存的 `fitness`，因此不会增加 `benchmark evaluation` 次数。

Mutation 对比实验。

`mutation` 对比实验固定 `crossover_type = pmx`，分别比较 `swap`、`insert`、`reversal`、`shuffle` 和 `shift` 五种 `mutation`。其余主要配置为 `selection_type = tournament`、`tournament_size = 3`、`correlation_threshold = 0.68`、`archive_size = 20`、`pop_size = 20`、`init_sampler_type = init_v3`。

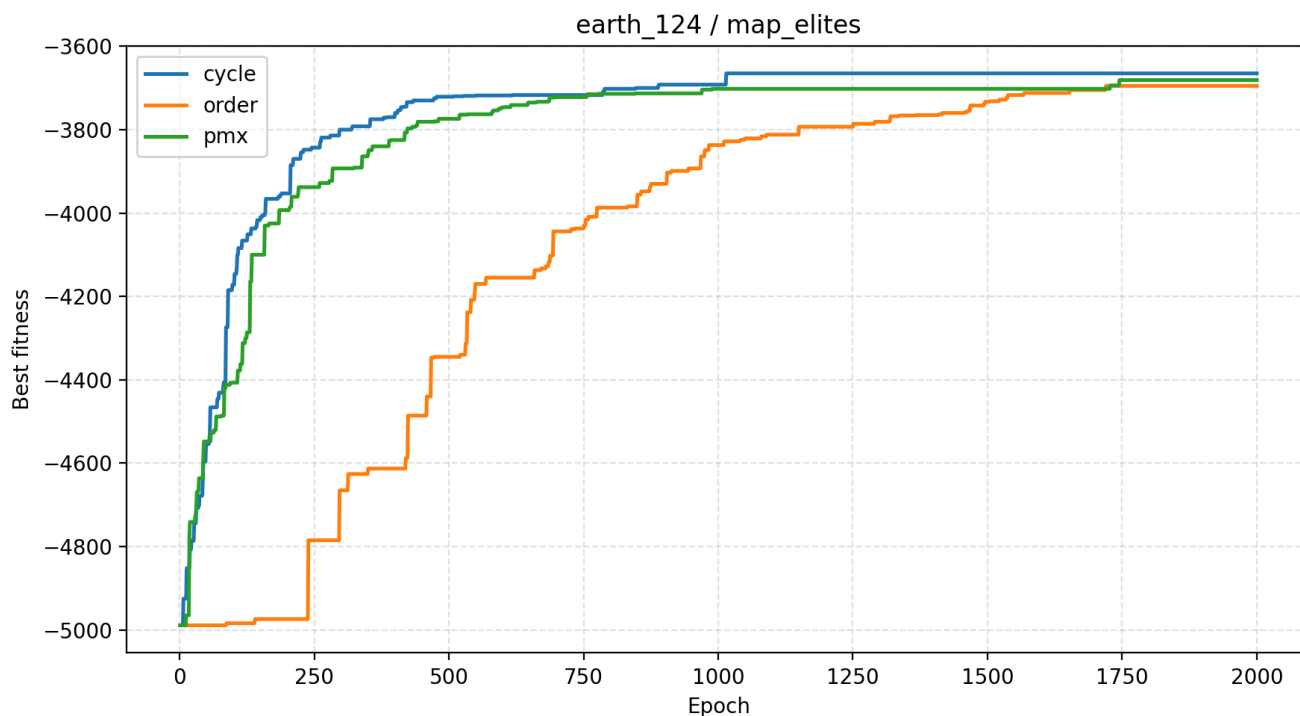


从最终结果看，`swap` 在该组实验中表现最好，最终达到 `-3666`。`insert` 排名第二，最终为 `-3678`，仍然是一个稳定的基础算子。`shift` 前期推进速度较快，但最终值为 `-3691`，后期改善幅度有限。`reversal` 和 `shuffle` 的最终结果相对靠后，主要原因可能是二者扰动较强，容易破坏已经形成的局部排序结构。

从收敛速度看，`insert` 和 `shift` 在前期推进较快，能够较早进入 `-3800` 附近；`swap` 前期略慢，但中后期持续改善，最终取得最好结果。这说明对于 `earth_124`，较温和的局部扰动在后期精修阶段更有优势，而强随机扰动不一定能带来更好的最终解。

Crossover 对比实验。

crossover 对比实验固定 `mutation_type = mix`（均匀随机选取 mutation 算子），分别比较 `order`、`pmx` 和 `cycle` 三种 crossover。其余配置与 mutation 对比实验保持一致，即仍然使用 MAP-Elites、锦标赛选择、`correlation_threshold = 0.68`、`archive_size = 20` 和 `init_sampler_type = init_v3`。



crossover	total evaluations	archive size	final y_best	排名
cycle	39964	20	-3665	1
pmx	39964	20	-3681	2
order	39964	20	-3695	3

从最终结果看，`cycle crossover` 表现最好，达到 `-3665`；`pmx` 排名第二，最终为 `-3681`；`order` 最终为 `-3695`。从曲线看，`cycle` 在前期和中期都比较领先，并且后期继续提升到本组最优值。`pmx` 整体较稳定，但后期突破能力弱于 `cycle`。`order` 更强调相对顺序继承，在本问题上对具体位置结构的保留可能不足，因此收敛速度和最终结果都不如前两者。

综合 mutation 和 crossover 对比可以看出，`earth_124` 对小幅度局部调整和位置结构继承较敏感。mutation 方面，`swap` 和 `insert` 更适合作为基础算子；crossover 方面，`cycle` 是本组最优选择。因此后续优化实验主要围绕 `cycle crossover` 展开。

2.3 演化算法的一些优化

在基础 MAP-Elites 框架上，我们进一步实现了若干改进，目标是提高搜索效率、增强跳出平台的能力，并更好地利用问题本身的约束结构。这些改进主要位于 `algorithms/_map_elites.py`、`algorithms/_constraint_utils.py`、`main.py` 和 `main_best.py`。

Adaptive Mix Mutation.

`adaptive_mix` 的核心思想是：不同 mutation 在搜索不同阶段的作用不同。前期需要更多探索，中期需要稳定推进，后期需要更细致的局部调整。因此我们不固定使用某一种 mutation，而是在 `swap`、`insert`、`reversal`、`shuffle` 和 `shift` 之间动态选择。

实现上，我们为每种 mutation 维护尝试次数和成功次数。如果某种 mutation 生成的子代成功进入或替换 archive，则认为它在近期更有效。最终选择概率由阶段性先验和历史成功率共同决定（`algorithms/_map_elites.py` 中的 `_select_adaptive_mutation()`）。这种方法不增加 evaluation 次数，因为每个子代仍然只评价一次。

该改进的预期作用是避免长期依赖单一 mutation，并让算法根据搜索过程自动调整扰动强度。它更偏向“算子层面的自适应选择”，不直接使用问题约束信息。

Adaptive Correlation 与 Local Search.

MAP-Elites 的 archive 更新依赖候选解与已有 archive 个体之间的 Kendall correlation，利用一个阈值，计算候选解与当前种群中解序列的相似程度，来决定候选解是否要加入种群，以增加种群的多样性。固定 `correlation_threshold` 会影响 archive 对多样性和质量替换的平衡。我们实现了动态 `correlation_threshold`：archive 尚未完成 warm-up 时使用原始阈值，避免前期 archive 填充被卡住；warm-up 后再逐步调整阈值，使搜索从较强调多样性逐渐转向质量替换。

在此基础上，我们还加入 local search，每隔固定 epoch 从当前 batch 中选择较好的候选解，生成少量 `insert/swap` 邻居并额外评价。其核心逻辑是：

```
1 top_indices = np.argsort(cands_y)[-top_k:]
2 local_cands = make_local_neighbors(local_seed_cands)
```

local search 的作用是加强后期 exploitation。当搜索已经找到较好区域后，对高质量候选做少量邻域搜索，可能更快找到附近的改进解。但它会增加额外 evaluation，因此需要控制触发频率和邻居数量。

Large Neighborhood Search Destroy-Repair (LNS)。

普通 mutation 通常只移动一个或少数事件，而 LNS 一次重构一小组 taxon 的 FAD/LAD 事件。实现时，先随机选择若干 taxon，删除它们的 FAD 和 LAD，再以满足 FAD 早于 LAD 的方式重新插回序列（`algorithms/_map_elites.py` 中的 `_lns_destroy_repair()`）。

LNS 的主要作用是提供比普通 mutation 更大的结构扰动，尤其适合中后期平台阶段。当普通 `insert` 或 `swap` 难以跳出当前局部结构时，LNS 可以重构一小片局部子问题，从而产生新的结构组合。

Threshold Accepting 与 Diversity Bonus.

MAP-Elites 的 archive 接收规则决定了哪些候选解可以保留下来。原始规则在候选解与 archive 个体过于相似时，通常要求候选 fitness 不低于被比较个体。我们加入 threshold accepting，允许前中期保留一部分略差但仍有潜力的候选：

```
1 fitness >= reference_fitness - threshold
```

其中 threshold 会随 epoch 逐渐衰减，后期重新回到更严格的质量选择。同时，我们加入 diversity bonus，根据候选解与 archive 中最相似个体的 Kendall correlation 计算结构差异。结构差异越大，前期获得的接收加分越高。该机制的目的不是直接生成更好的子代，而是让 archive 在早期保留更多结构多样性，为后续搜索提供更多可能路径。

Constraint Repair 与 Constraint Insert.

最后，我们利用 `coex.dat` 和 `Fb4L.dat` 中的领域约束设计了约束引导算子。`coex.dat` 表示 taxon 之间的共生关系，`Fb4L.dat` 表示 FAD/LAD 相关先后关系。我们将它们统一转换为 event-level precedence constraints：

```
1 event_a must appear before event_b
```

转换方式很直接：若 taxon `a` 与 `b` 共生，则要求 `FAD_a < LAD_b` 且 `FAD_b < LAD_a`；若 `Fb4L(a, b)` 成立，则要求 `FAD_a < LAD_b`。在 `earth_124` 上，最终得到 1906 条 event-level 约束。

`constraint_repair` 在子代评价前扫描违反的 precedence constraints，并通过插入操作将前驱事件移动到后继事件前面。`constraint_insert` 则在 mutation 时优先选择当前解中违反的约束进行修复；若没有违反约束，则退化为普通 `insert`。这两个方法引入了对这个问题的先验理解，不修改 benchmark，也不额外调用 fitness evaluation，因此是一种低成本的问题结构利用方式。

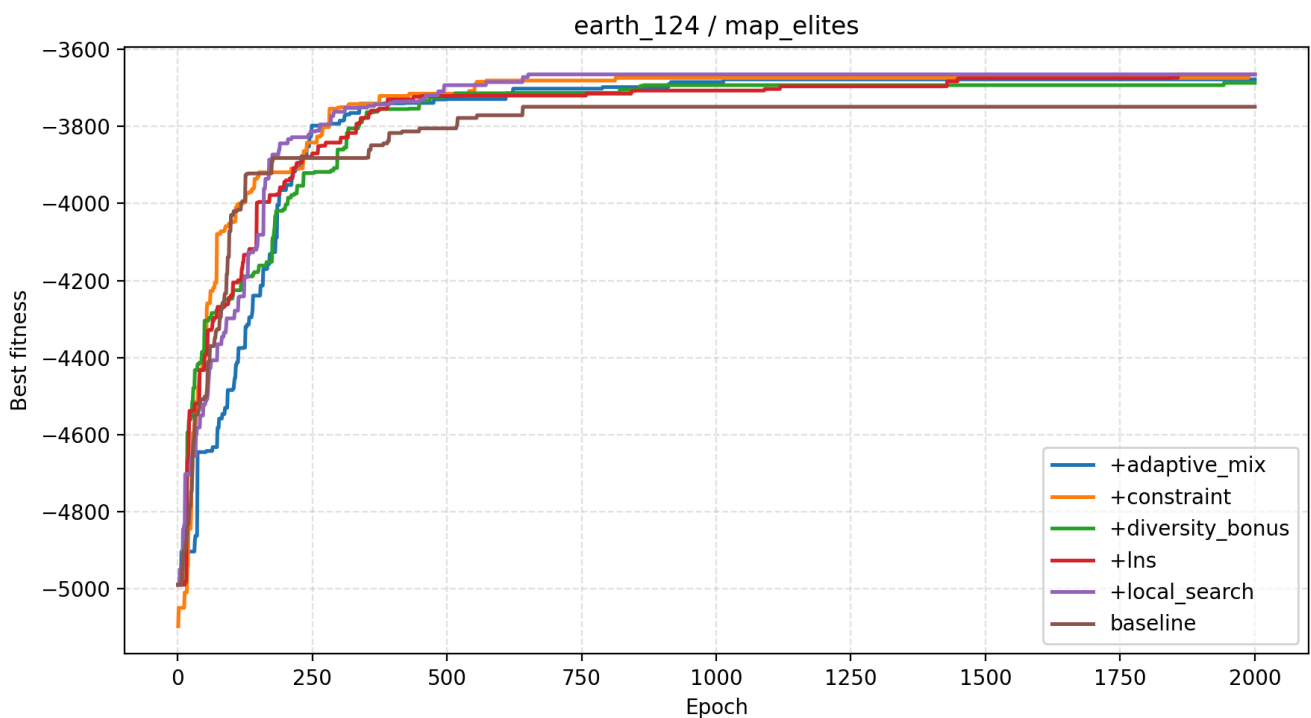
这些优化从不同角度改进演化算法：adaptive mix 改变基础算子选择，local search 强化局部精修，LNS 增强大邻域探索，threshold/diversity 改变 archive 接收策略，constraint 方法则直接利用问题约束信息。

2.4 寻找最优演化算法的实验

本部分实验集中在 `earth_124` 任务上。我们首先对单项优化进行消融比较，随后根据单项实验组合不同优化策略，尝试设计当前最优的演化算法。

单项优化实验。

单项实验对比曲线如下：



从曲线和最终结果看，`+constraint`、`+lns` 和 `+local_search` 都达到当前单项实验中的最好最终值 `-3665`，但它们的收敛过程不同。

`+constraint` 使用 `constraint_insert` + `constraint_repair`，不增加额外 evaluation。它在前期稳定推进，epoch 500 达到 `-3715`，后期仍然持续改善，直到 epoch 1989 达到 `-3665`。这说明约束引导不仅能减少明显违反 FAD/LAD 和共生关系的候选，也能在后期继续提供有方向的小步调整。

`+local_search` 达到 `-3665` 的速度最快，epoch 651 即达到最终值。它的优势是 exploitation 很强，可以围绕当前高质量候选快速精修。不过它额外使用了 160 次 local search evaluation，因此计算代价略高。

`+lns` 的前期速度不如 local search，但中后期持续改进明显。它在 epoch 1000 为 `-3707`，epoch 1500 为 `-3675`，最后在 epoch 1858 达到 `-3665`。这符合 LNS 的定位：它不是局部微调，而是在平台阶段通过较大的结构重构寻找新的组合。

`+adaptive_mix` 最终达到 `-3678`。它比基础配置有明显改善，但没有达到约束引导、LNS 和 local search 的最终值。这说明单纯用静态方法学习“哪个普通 mutation 更成功”仍然不如直接利用问题约束或引入大邻域搜索，后续我们还尝试使用强化学习得到更优秀的 mutation 算子。

`+diversity_bonus` 最终达到 `-3687`，说明放松 archive 接收门槛可以增加多样性，但如果子代生成本身没有更强的问题结构引导，其最终提升仍然有限。

目前最优算法设计。

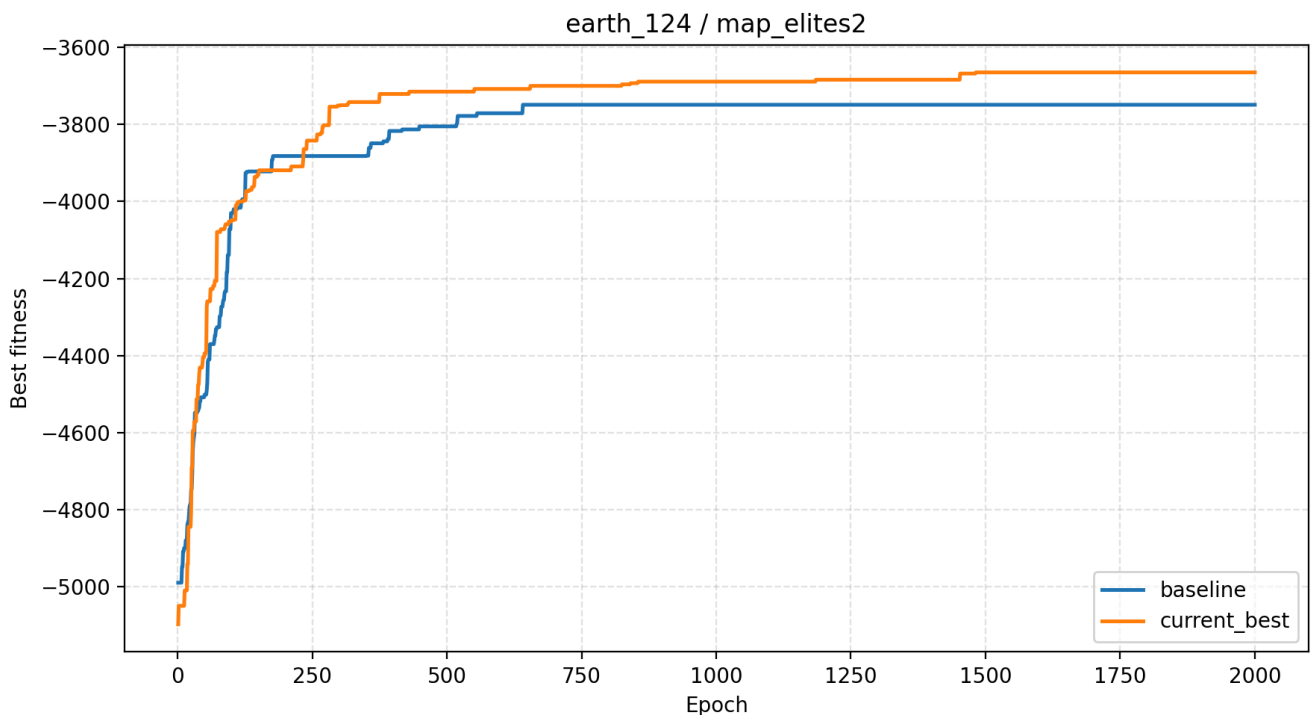
根据单项实验，我们发现不同优化在不同阶段的作用不同：`constraint` 方法适合前期建立较好的可行结构，并能在后期持续提供约束方向；`LNS` 适合中后期突破平台；`local search` 适合后期围绕高质量候选进行精修。根据充分利用每个优化方案的“强势期”，我们设计了分阶段策略：

```
1 Phase 1: epoch 1-500 (前期)
2   constraint_insert + constraint_repair
3
4 Phase 2: epoch 501-1500 (中期)
5   insert + constraint_repair + LNS
6
7 Phase 3: epoch 1501-2000 (后期)
8   insert + constraint_repair + LNS + local_search
```

这个设计的关键是按阶段分配搜索压力。前 500 个 epoch 使用 `constraint` 方法，让种群尽快向满足领域约束的区域靠近；中期保留 `constraint_repair`，同时加入 `LNS`，让搜索在较好结构基础上进行大邻域重构；后期再加入少量 `local search`，用较少额外 evaluation 做局部精修。`constraint_repair` 全程保留，是因为它不增加 benchmark evaluation，并且能持续把候选解拉回更合理的约束结构。

Best 算法实验结果。

best 算法与 baseline 的对比曲线如下：



从结果看，current best 最终达到 `-3665`，与单项实验中的 `+constraint`、`+lns` 和 `+local_search` 并列为当前最好最终值。它的总评价次数为 40004，其中 `local search` evaluation 只有 40 次。相比单独开启 `local search` 的 160 次额外评价，分阶段策略以更少额外评价达到同样最终值。

从收敛过程看，current best 的行为符合预期。Phase 1 结束时，算法已经达到 -3715，说明 constraint 阶段为后续搜索提供了较好的结构基础。Phase 2 中加入 LNS 后，曲线继续从 -3715 改进到 -3665，说明中期大邻域扰动确实发挥了跳出平台和重组局部结构的作用。Phase 3 中加入 local search 后，最优值保持稳定，说明后期精修没有破坏已有 elite，同时提供了额外的局部搜索保障。

best 算法之所以 work，是因为它的有效性来自三个方面：第一，利用先验数据集的约束信息减少无效搜索；第二，通过 LNS 在平台期提供比普通 mutation 更大的结构变化；第三，只在后期使用 local search，避免过早陷入相似局部结构，同时控制额外 evaluation 数量。这种分阶段的策略比简单叠加所有机制更清晰，也更符合演化搜索从探索到精修的过程。

三、基于强化学习的演化算法

3.1 可学习演化算子的设计与实现

传统演化算法中的 mutation 和 crossover 通常由人工规则定义，例如随机交换、插入、片段交叉等。这些算子实现简单、计算代价低，但它们并不会主动学习当前问题中哪些事件顺序更有潜力。adaptive_mix 根据算法运行过程中算子的成功率，动态调整选择每个算子的概率，虽然有比 baseline 更好的表现，但并不是最优。因此我们尝试使用基于强化学习的演化算子：让一个 policy 从已有父代和搜索历史中学习如何构造更好的子代，从而把“生成候选解”本身变成一个可学习过程。

RLEA 的基本推理流程为：

- 1 选择父代 `x1, x2`
- 2 将父代及其 `fitness` 输入 RL 环境
- 3 `Q network` 根据当前 `partial sequence` 选择 `action`
- 4 环境执行 `action` 并更新部分子代
- 5 直到构造出完整 `offspring`

在训练过程中，模型通过环境交互学习如何从父代信息构造子代；在推理过程中，`main.py` 根据 `r1_freq` 周期性启用 RL operator，其余 epoch 仍然使用传统 EA 算子。

3.2 可学习演化算子的训练推理实验

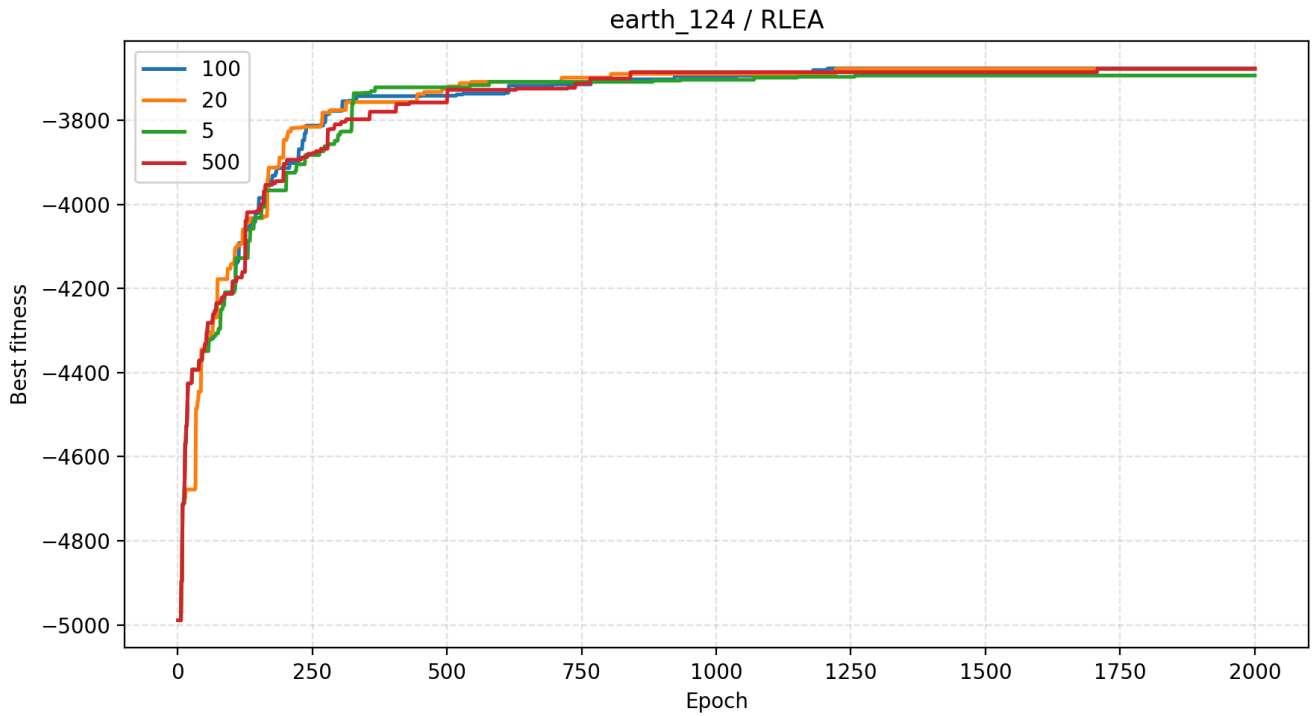
RLEA 推理前需要先训练 policy checkpoint。训练前还需要准备 RL 环境使用的父代池，因此我们新增了 `scripts/generate_r1_parents.py`，用于生成 `data/earth_124/gen_points_v3/` 下的父代样本。训练命令如下：

```
1 python train.py --task_name EarthBenchEnv_124 --epoch 2000 \  
2   --train_envs 100 --test_envs 100 \  
3   --save_path multi_envs_models/dim124_epoch2000.pth \  
4   --device cuda:0
```

训练完成后，推理运行脚本为 `scripts/run_earth_124_r1ea.sh`。我们还加入了 `r1_freq` 参数，用来控制每隔多少 epoch 使用一次 RL operator：

- 1 `r1_freq = 500` 表示每 500 个 epoch 启用一次 RL
- 2 `r1_freq = 100` 表示每 100 个 epoch 启用一次 RL
- 3 `r1_freq = 20` 表示每 20 个 epoch 启用一次 RL
- 4 `r1_freq = 5` 表示每 5 个 epoch 启用一次 RL

不同 `r1_freq` 的实验结果，对比曲线如下：



从曲线看，四组实验前期收敛趋势接近，都能较快从约 -5000 提升到 -37xx 区间；主要差异出现在中后期平台阶段。`r1_freq=500` 时，2000 epoch 内只启用约 4 次 RL operator，RL 对整体搜索的影响较弱，因此最终结果与传统 EA 很接近。`r1_freq=20` 的中期推进较快，但后期没有继续突破，最终与低频设置接近。`r1_freq=5` 使用 RL 过于频繁，policy 介入过强，传统 EA 的随机探索空间被压缩，因此最终结果反而较低，并且频繁使用 RL 推理，算法的运行时间显著增加。

综合本组实验，`r1_freq=100` 是相对最优设置，最终达到 `y_best = -3677`。它在 2000 epoch 内约启用 20 次 RL operator，既能让学习到的子代构造策略周期性参与搜索，又不会完全替代传统 EA 的 crossover/mutation，算法运行的延迟较小。这个结果说明，当前训练得到的 RL operator 更适合作为周期性辅助算子，而不是每一轮都强制使用。适度频率可以在学习型重组和传统随机探索之间取得较好平衡。

四、总结

本次实验围绕生物多样性曲线优化问题，分别实现和比较了传统 MAP-Elites 演化算法与基于强化学习的 RLEA 算法。

在基础演化算子比较中，mutation 实验表明 `swap` 和 `insert` 这类小扰动算子更适合这种事件排序的序列优化问题；crossover 实验表明 `cycle crossover` 表现最好，说明保留父代中的位置结构对本问题较有帮助。在传统 MAP-Elites 优化中，单纯调整基础 mutation 概率的收益有限，更有效的方向是利用问题结构和分阶段搜索策略。我们实现的 `constraint_insert + constraint_repair` 直接利用问题的先验约束信息，在不增加 evaluation 的情况下达到较好结果；LNS 在中后期提供大邻域结构扰动；local search 在后期提供精修能力。最终设计的 phased best 算法

按照“前期约束引导、中期大邻域重构、后期局部精修”的思路组织这些模块，在较少额外 evaluation 下达到当前最优结果 -3665。

在强化学习部分，我们使用 RLEA 将子代构造过程交给可学习 policy 完成，并通过 `rl_freq` 控制 RL operator 的使用频率。实验表明，RL operator 不宜过于频繁地替代传统 EA；`rl_freq=100` 在本组实验中效果最好，说明周期性使用学习型算子可以作为传统演化搜索的补充。

总体来看，对算法的优化要基于对问题的理解，把问题约束、搜索阶段和算子特性结合起来，才能得到针对特定问题的最优算法。